1. Project Title

Banking Data Governance & Data Quality Pipeline using PySpark, SQL, Python, and OpenMetadata

2. Project Purpose

This project is designed as a hands-on practice lab to strengthen skills required for a Data Governance / Collibra / Data Quality Data Engineer role below:

Job Description:

- Data-Collibra with Data Governance,Data Catalog, Data Lineage,Data Quality.
- Skills from outside primary skill cluster SQL, PySpark, Python Scripting, Informatica or Collibra, JIRA, Agile, Scrum
- Must have a minimum of 4+ years of experience in data engineering, data management, or data governance roles.
- Must have worked in Agile development environment and familiar with Software Development Lifecycle, Agile/Scrum process.
- Strong hands-on experience with SQL, PySpark and Python for data manipulation and automation.
- Experience with enterprise governance tools such as Collibra, Informatica, etc.
- Solid understanding of data modeling, data warehousing concepts, and schema design

The goal is to practice the actual technical and governance skills behind the job description so that I can confidently speak about my real work experience in data governance, data cataloging, data lineage, data quality, SQL, PySpark, Python scripting, Agile/Scrum, data modeling, and data warehousing.

The project will simulate a banking data governance environment where **synthetic customer, account, transaction, branch, product, risk-score, reference-code, data-quality-rule, stewardship-issue, and source-to-target mapping data are generated, stored, processed, validated, cataloged, governed, and monitored.**

Since Collibra is a closed-source enterprise platform, this project will use **OpenMetadata** as an open-source governance/catalog tool to practice similar concepts such as data catalog, business glossary, ownership, data assets, metadata, classifications, and lineage.

3. Target Job Description Alignment

The project is designed to practice the following job description requirements:

- Data Governance
- Data Catalog
- Data Lineage
- Data Quality
- SQL
- PySpark

- Python Scripting
- Enterprise governance tool concepts such as Collibra / Informatica
- JIRA
- Agile
- Scrum
- SDLC
- Data Modeling
- Data Warehousing
- Schema Design

4. High-Level Architecture

The project architecture will follow this flow:

Faker Data Generator
→ PostgreSQL Banking Source System
→ Python Extraction / Ingestion Layer
→ Raw Zone using Local Parquet or MinIO
→ PySpark Processing Layer
→ Curated Warehouse Layer
→ SQL + PySpark Data Quality Layer
→ OpenMetadata Governance Layer
→ Stewardship / Issue Tracking / Agile Evidence Layer

5. Architecture Description

5.1 Data Generation Layer

The project will start with a Faker-based synthetic data generator.

The generator will create realistic banking data such as:

- Customers
- Accounts
- Transactions
- Branches
- Products
- Customer risk scores

Example fields:

- customer_id
- customer_name
- email
- phone_number
- date_of_birth
- account_id
- account_type
- account_status
- transaction_id
- transaction_amount
- transaction_type
- transaction_date
- branch_id
- product_id
- risk_score

Purpose:

This layer helps simulate real banking source data and gives practice with Python scripting, schema design, and banking-domain data modeling.

JD alignment:

- Python scripting
- Data modeling
- Schema design
- Data engineering
- Banking domain understanding

5.2 PostgreSQL Source System Layer

PostgreSQL will act as the operational banking source system.

Source tables may include:

- src_customers

- src_accounts
- src_transactions
- src_branches
- src_products
- src_customer_risk_scores

Purpose:

This layer simulates an OLTP banking source system. It gives practice with SQL, source-system design, and source schema understanding.

JD alignment:

- SQL
- Data management
- Schema design
- Source-system understanding
- Data engineering experience

5.3 Python Extraction / Ingestion Layer

Python scripts will extract data from PostgreSQL and land it into the raw data zone.

Example scripts:

- extract_customers.py
- extract_accounts.py
- extract_transactions.py
- extract_branches.py
- extract_products.py
- extract_all_sources.py

Expected output:

- data/raw/customers/
- data/raw/accounts/
- data/raw/transactions/
- data/raw/branches/

- data/raw/products/
- data/raw/customer_risk_scores/

Purpose:

This layer gives hands-on practice with Python automation, data extraction, file creation, logging, and basic data movement.

JD alignment:

- Python scripting
- Data manipulation
- Automation
- Data ingestion
- SDLC-style pipeline development

5.4 Raw Data Zone

Raw source extracts will be stored as Parquet files.

Initial implementation can use local folders:

- data/raw/customers/
- data/raw/accounts/
- data/raw/transactions/
- data/raw/branches/
- data/raw/products/
- data/raw/customer_risk_scores/

Later, this can be extended to MinIO object storage.

Purpose:

This layer represents the raw landing zone. It preserves source data before cleansing and transformation.

JD alignment:

- Data warehousing concepts
- Raw layer design
- Data lineage
- Data catalog

- Metadata management
- Schema design

5.5 PySpark Processing Layer

PySpark will process the raw data and create cleaned, standardized, curated outputs.

Example Spark jobs:

- process_customers.py
- process_accounts.py
- process_transactions.py
- process_branches.py
- process_products.py
- build_customer_risk_profile.py
- build_curated_models.py

PySpark transformations may include:

- Schema enforcement
- Null handling
- Standardizing customer names
- Standardizing account status values
- Deduplicating customer records
- Joining transactions with accounts
- Joining accounts with customers
- Creating customer risk bands
- Calculating daily transaction totals
- Building curated warehouse-style outputs

Purpose:

This layer gives hands-on practice with PySpark for data manipulation, cleansing, transformation, joins, validations, and curated dataset creation.

JD alignment:

- PySpark

- Data manipulation
- Automation
- Data quality
- Data engineering
- Schema design
- Data warehousing

5.6 Curated Warehouse Layer

The curated layer will contain warehouse-style tables designed for reporting, analytics, and governance practice.

Proposed curated tables:

- dim_customer
- dim_account
- dim_branch
- dim_product
- fact_transaction
- fact_customer_risk_score
- fact_daily_account_balance

For each table, the project will document:

- Business purpose
- Grain
- Primary key
- Foreign keys
- Source tables
- Target columns
- Critical Data Elements
- Data quality rules
- Owner
- Steward

- Lineage

Example:

Table: fact_transaction
Grain: One row per transaction
Primary key: transaction_id
Foreign keys: customer_id, account_id, branch_id, product_id
Purpose: Supports transaction-level analysis, reconciliation, fraud checks, and financial reporting.

Purpose:

This layer gives practical experience with dimensional modeling, fact/dimension design, schema design, data warehousing, and governed curated datasets.

JD alignment:

- Data modeling
- Data warehousing
- Schema design
- SQL
- Governance-ready data design

5.7 SQL Validation Layer

SQL scripts will be created to validate data quality in source, raw, and curated layers.

Folder:

sql/validation/

Example scripts:

- 01_null_checks.sql
- 02_duplicate_checks.sql
- 03_accepted_values_checks.sql
- 04_referential_integrity_checks.sql
- 05_source_target_reconciliation.sql
- 06_control_totals.sql
- 07_exception_records.sql

Example validations:

- customer_id must not be null
- transaction_id must be unique
- account_status must be ACTIVE, CLOSED, or SUSPENDED
- transaction_amount must not be null
- transaction_amount must be greater than zero
- transactions must map to valid accounts
- accounts must map to valid customers
- source row count should match target row count
- source transaction total should match curated transaction total

Purpose:

This layer gives hands-on SQL practice for real data quality and reconciliation scenarios.

JD alignment:

- SQL
- Data quality
- Data manipulation
- Source-to-target reconciliation
- Data warehousing
- Production-style validation

5.8 PySpark Data Quality Layer

PySpark scripts will also be used to perform large-scale profiling and validation.

Folder:

spark/dq/

Example scripts:

- profile_customers.py
- profile_accounts.py
- profile_transactions.py
- validate_customer_quality.py
- validate_account_quality.py

- validate_transaction_quality.py
- generate_dq_summary.py

Example PySpark checks:

- Record count
- Schema validation
- Null count by column
- Duplicate primary keys
- Accepted values
- Transaction amount range checks
- Orphan transaction checks
- Source-to-target mismatch detection

Expected outputs:

- data/dq_results/dq_summary.json
- data/dq_results/dq_exceptions.parquet
- data/dq_results/rule_results.csv

Purpose:

This layer provides hands-on practice with PySpark-based data profiling, validation, and data quality reporting.

JD alignment:

- PySpark
- Data quality
- Python automation
- Large-scale data validation
- Data manipulation

5.9 Python Governance Automation Layer

Python scripts will automate metadata and governance-related tasks.

Folder:

python/automation/

Example scripts:

- extract_schema_metadata.py
- generate_data_catalog.py
- parse_dq_rules.py
- generate_lineage_report.py
- create_stewardship_issues.py
- generate_governance_summary.py

Automation examples:

- Extract schema metadata from source and curated tables
- Generate catalog YAML entries
- Parse data quality rules
- Read DQ results
- Create data quality issue records
- Generate lineage reports
- Create governance summary output

Purpose:

This layer connects data engineering with governance automation.

JD alignment:

- Python scripting
- Automation
- Metadata management
- Data catalog
- Data lineage
- Data quality
- Governance reporting

5.10 OpenMetadata Governance Layer

OpenMetadata will be used as the open-source governance and catalog platform.

OpenMetadata will help practice concepts similar to Collibra:

- Data catalog
- Business glossary
- Data assets
- Ownership
- Stewardship concepts
- Metadata management
- Tags
- Classifications
- Data lineage
- Data discovery

Example glossary terms:

- Customer
- Account
- Transaction
- Transaction Amount
- Customer Identifier
- Account Balance
- Customer Risk Score
- Critical Data Element

Example data assets:

- PostgreSQL.src_customers
- PostgreSQL.src_accounts
- PostgreSQL.src_transactions
- data.raw.customers
- data.raw.accounts
- data.raw.transactions
- curated.dim_customer
- curated.dim_account
- curated.fact_transaction

Example tags/classifications:

- PII
- Confidential
- Critical Data Element
- Customer Data
- Payments Data
- Risk Data

Purpose:

This layer provides hands-on practice with enterprise data governance tool concepts using an open-source tool.

JD alignment:

- Enterprise governance tool concepts
- Collibra-style learning
- Data governance
- Data catalog
- Data lineage
- Metadata management

Important note:

This project will not claim to use Collibra. It will use OpenMetadata as an open-source counterpart to practice similar governance concepts.

5.11 Manual Governance Artifacts Layer

In addition to OpenMetadata, the project will maintain manual YAML and Markdown governance artifacts.

Folder:

governance/

Proposed files:

- governance/glossary/business_terms.yml
- governance/catalog/data_assets.yml
- governance/metadata/critical_data_elements.yml

- governance/metadata/source_to_target_mapping.yml
- governance/lineage/lineage_mapping.yml
- governance/data_quality/dq_rules.yml
- governance/stewardship/data_quality_issues.yml
- governance/governance_policy.md

Purpose:

These files help manually practice Collibra-style governance thinking before and alongside OpenMetadata implementation.

JD alignment:

- Data governance
- Data catalog
- Business glossary
- Data lineage
- Data quality
- Metadata management
- Data stewardship
- Documentation
- Communication

5.12 Agile / JIRA / Scrum Simulation Layer

The project will simulate JIRA-style delivery using Markdown and YAML files.

Folder:

agile/

Example files:

- agile/sprint_backlog.yml
- agile/user_stories.md
- agile/definition_of_done.md
- agile/test_evidence.md
- agile/release_notes.md

Example user story:

As a Data Steward, I want Transaction Amount to be defined as a Critical Data Element so that downstream reporting teams can apply quality rules and ownership controls.

Example acceptance criteria:

- Transaction Amount is added to the business glossary
- Transaction Amount is marked as a Critical Data Element
- Related data assets are cataloged
- Data lineage is mapped from source to curated layer
- SQL validation rule is created
- PySpark validation rule is created
- Failed records are logged as stewardship issues
- Test evidence is documented

Purpose:

This layer helps practice Agile/Scrum and SDLC delivery for governance and data quality work.

JD alignment:

- JIRA
- Agile
- Scrum
- SDLC
- Communication
- Team collaboration
- Requirement-to-release delivery

6. Proposed Repository Structure

banking-data-governance-pipeline/

- README.md
- docker-compose.yml
- requirements.txt

data-generator/

- generate_customers.py

- generate_accounts.py
- generate_transactions.py
- generate_branches.py
- generate_products.py
- generate_customer_risk_scores.py

postgres/

- init.sql
- schema.sql
- seed_data.sql

python/

- ingestion/
    - extract_customers.py
    - extract_accounts.py
    - extract_transactions.py
    - extract_all_sources.py
- automation/
    - extract_schema_metadata.py
    - generate_data_catalog.py
    - parse_dq_rules.py
    - generate_lineage_report.py
    - create_stewardship_issues.py

spark/

- jobs/
    - process_customers.py
    - process_accounts.py
    - process_transactions.py
    - process_branches.py
    - process_products.py
    - build_curated_models.py

- dq/
    - profile_customers.py
    - profile_accounts.py
    - profile_transactions.py
    - validate_customer_quality.py
    - validate_account_quality.py
    - validate_transaction_quality.py
    - generate_dq_summary.py

sql/

- validation/
    - 01_null_checks.sql
    - 02_duplicate_checks.sql
    - 03_accepted_values_checks.sql
    - 04_referential_integrity_checks.sql
    - 05_source_target_reconciliation.sql
    - 06_control_totals.sql
    - 07_exception_records.sql

governance/

- README.md
- governance_policy.md
- glossary/
    - business_terms.yml
- catalog/
    - data_assets.yml
- metadata/
    - critical_data_elements.yml
    - source_to_target_mapping.yml
- lineage/
    - lineage_mapping.yml

- data_quality/
    - dq_rules.yml
- stewardship/
    - data_quality_issues.yml

openmetadata/

- README.md
- ingestion_configs/
    - postgres_metadata_ingestion.yml
    - curated_layer_metadata_ingestion.yml
    - lineage_ingestion.yml

agile/

- sprint_backlog.yml
- user_stories.md
- definition_of_done.md
- test_evidence.md
- release_notes.md

data/

- raw/
- curated/
- dq_results/

docs/

- architecture.md
- jd_alignment.md
- learning_notes.md

7. JD-to-Project Mapping

JD Requirement	**Project Implementation**

Data-Collibra	OpenMetadata will be used as an open-source counterpart to practice Collibra-style governance concepts
Data Governance	Governance policy, business glossary, owners, stewards, CDEs, classifications, stewardship issues
Data Catalog	OpenMetadata catalog and governance/catalog/data_assets.yml
Data Lineage	governance/lineage/lineage_mapping.yml and OpenMetadata lineage
Data Quality	SQL validation, PySpark validation, dq_rules.yml, DQ results, issue tracking
SQL	PostgreSQL queries and sql/validation scripts
PySpark	Spark processing jobs and PySpark DQ scripts
Python Scripting	Data generation, ingestion, metadata extraction, catalog generation, DQ automation
Informatica or Collibra	OpenMetadata used to practice comparable enterprise governance concepts
JIRA	Agile/JIRA-style files such as sprint backlog, user stories, test evidence, release notes
Agile	Sprint-based project execution
Scrum	User stories, acceptance criteria, definition of done, sprint backlog
4+ years data engineering/governance	Reinforces existing professional experience through practical implementation
SDLC	Requirement → design → build → validate → release → support
Data Modeling	Fact and dimension design
Data Warehousing	Curated warehouse layer
Schema Design	Source and target table design, keys, relationships, constraints
Communication	Business glossary, README, documentation, stewardship notes, test evidence

8. Project Phases

Phase 1: Repository Copy and Cleanup

Goal:

Create a new copy of the existing banking pipeline repository and rename it to:

banking-data-governance-pipeline

Tasks:

- Copy existing repository
- Rename project title
- Remove or de-emphasize dbt
- Update README to reflect governance and data quality focus
- Keep useful data generator logic
- Keep PostgreSQL source setup if useful
- Remove Power BI/dbt-first positioning from documentation

Deliverables:

- New repository
- Updated README
- Clean project structure

Phase 2: Banking Source Schema and Data Generator

Goal:

Create realistic synthetic banking source data.

Tasks:

- Define PostgreSQL source schema
- Generate customers
- Generate accounts
- Generate transactions
- Generate branches
- Generate products
- Generate customer risk scores
- Load generated data into PostgreSQL

Deliverables:

- PostgreSQL source tables

- Faker-based data generator
- Sample banking data

Phase 3: Python Ingestion Layer

Goal:

Extract source data from PostgreSQL into raw Parquet files.

Tasks:

- Build Python extraction scripts
- Connect to PostgreSQL
- Extract each source table
- Write raw files to data/raw/
- Add logging and error handling
- Add load date partitioning

Deliverables:

- Python ingestion scripts
- Raw Parquet files
- Ingestion logs

Phase 4: PySpark Processing Layer

Goal:

Use PySpark to clean, standardize, transform, and model banking data.

Tasks:

- Read raw Parquet files
- Apply schema validation
- Clean customer data
- Clean account data
- Clean transaction data
- Join customer, account, transaction, branch, product, and risk-score data
- Build curated fact and dimension outputs

Deliverables:

- PySpark processing jobs
- Curated Parquet outputs
- Warehouse-style tables

Phase 5: Data Modeling and Schema Design

Goal:

Design curated fact and dimension models.

Tasks:

- Define dim_customer
- Define dim_account
- Define dim_branch
- Define dim_product
- Define fact_transaction
- Define fact_customer_risk_score
- Define fact_daily_account_balance
- Document grain, keys, relationships, and business purpose

Deliverables:

- Curated data model
- Data model documentation
- Schema design documentation

Phase 6: SQL Data Quality Validation

Goal:

Practice SQL-based data validation and reconciliation.

Tasks:

- Write null-check queries
- Write duplicate-check queries
- Write accepted-values checks

- Write referential-integrity checks
- Write source-to-target reconciliation queries
- Write control-total checks
- Write exception-record queries

Deliverables:

- SQL validation scripts
- Data quality result examples
- Reconciliation outputs

Phase 7: PySpark Data Quality Validation

Goal:

Practice PySpark-based profiling and validation.

Tasks:

- Profile customers
- Profile accounts
- Profile transactions
- Validate schemas
- Validate null counts
- Validate duplicate keys
- Validate accepted values
- Validate amount ranges
- Generate DQ summary report

Deliverables:

- PySpark DQ scripts
- DQ summary JSON
- DQ exception files

Phase 8: Governance Artifacts

Goal:

Build Collibra-style governance artifacts manually.

Tasks:

- Create business glossary
- Create data catalog YAML
- Define critical data elements
- Create source-to-target mapping
- Create lineage mapping
- Create data quality rule catalog
- Create stewardship issue log
- Create governance policy

Deliverables:

- business_terms.yml
- data_assets.yml
- critical_data_elements.yml
- source_to_target_mapping.yml
- lineage_mapping.yml
- dq_rules.yml
- data_quality_issues.yml
- governance_policy.md

Phase 9: OpenMetadata Integration

Goal:

Use OpenMetadata to practice open-source data catalog and governance concepts.

Tasks:

- Install or configure OpenMetadata
- Register PostgreSQL source
- Register curated datasets
- Add glossary terms
- Assign owners

- Add tags/classifications
- Explore lineage
- Document OpenMetadata usage

Deliverables:

- OpenMetadata setup notes
- Metadata ingestion configuration
- Cataloged assets
- Glossary terms
- Ownership and classification examples

Phase 10: Agile / SDLC Simulation

Goal:

Practice how governance work is delivered through Agile/Scrum.

Tasks:

- Create user stories
- Create sprint backlog
- Define acceptance criteria
- Define definition of done
- Document test evidence
- Create release notes
- Track data quality issues as stewardship items

Deliverables:

- user_stories.md
- sprint_backlog.yml
- definition_of_done.md
- test_evidence.md
- release_notes.md

9. Learning Outcomes

After completing this project, I should be able to confidently explain and practice:

- What data governance means in a banking environment
- How a data catalog is structured
- How business glossary terms connect to technical data assets
- How data lineage is documented from source to curated layer
- How critical data elements are identified
- How data quality rules are defined and validated
- How SQL is used for validation and reconciliation
- How PySpark is used for data profiling and quality checks
- How Python is used for data ingestion and governance automation
- How OpenMetadata concepts map to Collibra-style governance concepts
- How Agile/Scrum delivery works for governance tasks
- How data modeling supports governed analytics
- How warehouse-style fact/dimension tables are designed

10. Success Criteria

The project will be considered successful when the following are completed:

1. Synthetic banking data is generated using Faker.
2. Source data is loaded into PostgreSQL.
3. Python scripts extract source data into raw Parquet.
4. PySpark jobs process raw data into curated warehouse-style outputs.
5. Curated fact and dimension tables are created.
6. SQL validation scripts are written for nulls, duplicates, accepted values, referential integrity, reconciliation, and control totals.
7. PySpark validation scripts generate DQ results.
8. Governance artifacts are created for glossary, catalog, CDEs, lineage, source-to-target mapping, DQ rules, and stewardship issues.
9. OpenMetadata is used to practice catalog, glossary, ownership, classification, and lineage concepts.
10. Agile/Scrum artifacts are created to simulate JIRA-style delivery.

11. Documentation clearly maps the project to the target JD skills.

11. Personal Motivation

This project is a hands-on practice lab to bridge the gap between professional experience and interview confidence.

The purpose is to make data governance, Collibra-style concepts, data catalog, lineage, data quality, SQL, PySpark, Python scripting, Agile/Scrum, and data modeling feel practical and natural.

By completing this project, I will be able to speak more confidently about how governance connects to real data engineering work:

- A business term becomes a glossary entry.
- A glossary term maps to technical data assets.
- A data asset has an owner and steward.
- A critical field becomes a Critical Data Element.
- A CDE gets data quality rules.
- A data quality rule becomes SQL and PySpark validation.
- A failed validation becomes a stewardship issue.
- A source-to-target path becomes lineage.
- A change request becomes an Agile user story.
- A fix gets test evidence and release notes.

This is the real workflow behind the job description.